r_tutorial_week_03

Werkcollege 3

Sebastian A. J. Lortz

15 October 2026

Table of contents

How to use this tutorial **2**

1 Start workflow **2**
Your turn . . . 3

2 From sample to population **3**
Your turn . . . 4

3 Sampling fluctuation **4**
Your turn . . . 5

4 The sampling distribution **6**
Your turn . . . 8

5 Confidence intervals by hand **8**
5.1 Visualisation . . . 9
5.2 Building the interval step by step . . . 10
Your turn . . . 12
5.3 Other confidence levels . . . 13
Your turn . . . 15

6 A confidence interval for your own variable **15**
Your turn . . . 17

7 Does the interval cover the true mean? **17**
Your turn . . . 18

8 Simulating many intervals **18**
Your turn . . . 20

How to use this tutorial

Welcome back! In Week 2 you learned to describe **one sample**: you loaded the class data, cleaned it, and summarised it with the mean, the standard deviation, histograms, and boxplots.

This week we take the next step. We have collected a **sample**, but we usually want to say something about a much larger **population**. For example, our class is a small sample, but we may want to say something about all first-year students. This week is about that step: **from sample to population**.

Work through this page carefully, from top to bottom. Every short explanation is followed by a **Your turn** task to do yourself.

General tips:

- Read each part slowly and in order. Each step builds on the one before it.
- Type the code yourself instead of copying and pasting it. You learn faster this way.
- Errors are normal. Read the message and try again.
- If you are stuck, check the FAQ or ask a teacher.
- Save your work often.

---

1 Start workflow

We begin, as always, with the **start workflow**. This week there is one extra step: we load the cleaned data you saved at the end of Week 2.

1. Open **RStudio**: click on the RStudio application.
2. Set the **working directory**: click *Session* > *Set Working Directory* > *Choose Directory*, and choose your `research_methods` folder.
3. Open a **new script**: click *File* > *New File* > *R Script*.
4. On the first line, write a comment with the title, for example: `# R tutorial - Week 3`.
5. Save the script as `week3.R` in your `research_methods` folder.
6. Load your cleaned data from Week 2 into an object called `survey`:

```
survey <- read.csv("class_survey_clean.csv")
```

> **Watch out**
>
> If you see `Error: cannot open file ...`, R cannot find the file. Check that your working directory is `research_methods` (run `getwd()` to print it), and that `class_survey_clean.csv` really sits in that folder. If you do not have the file, go back to the end of the Week 2 tutorial and save it first.

Your turn

Complete the **start workflow** and load `survey`. Check that `survey` appears in the **Environment** pane (top-right).

---

2 From sample to population

A **population** is the whole group you want to say something about, for example *all first-year Orthopedagogy students*. A **sample** is the smaller group you actually collected data from, for example *the students in our class*. We almost never sample a whole population, because it is too big. So we collect a sample and use it to make an estimate about the population.

When we compute the average of a sample, that number is a **point estimate** of the population average. A point estimate is our single best guess for the population value, based on the sample.

```
mean(survey$self_efficacy)   # point estimate of the population's average self-efficacy
#> [1] 33.23333
```

This one number is our best guess for the average self-efficacy of the whole population, based only on the information we collected in our class.

> **Example: what is the population?**
>
> Imagine you assessed the average number of study hours in our class and found 14 hours per week.
>
> - Your **sample** is our class.
> - Your **point estimate** is 14 hours.
> - Your **population** might be "all first-year students at this university", or "all Dutch first-year students", or something else.

> The point estimate is always the same number (14), but *what it says something about* depends on which population you have in mind.

Choosing your population is not a technical step; it is a thinking step. It depends on who you assessed and thus, if your sample generalises to the population.

Your turn

Think about your own research question (the one you will work with for the rest of the course).

1. Write down, in one sentence, what your **population** is.
2. Write down what your **sample** is.
3. Explain in one or two sentences why your sample may, or may not, be a good representation for that population.

---

3 Sampling fluctuation

If you took a **different** sample from the same population, you would get a slightly **different** point estimate. This is called **sampling fluctuation**: the sample mean jumps around a little from sample to sample, just by chance.

We can see this with our own data by taking **subsamples**. A **subsample** is a smaller, random part of your dataset. If our class is the sample, a subsample is a random handful of those students.

To take one, we need two new functions:

- `sample()` picks random numbers. We use it to pick random **row numbers**.
- `set.seed()` fixes the random choice, so it is the **same every time** you run it. This keeps your work **reproducible**: you and a classmate get the exact same "random" rows.

We then keep those rows with square brackets, `survey[rows, ]`, using the row indexing you learned in Week 2 (the empty part after the comma means "keep all columns").

> **i** Example: why `set.seed()` matters

Run these two lines, then run them again:

```
set.seed(1)
sample(1:30, 5)   # five random numbers between 1 and 30
```

With `set.seed(1)` in front, you get the **same** five numbers every time. Without it, you would get different numbers on each run, and your results would not be reproducible.

```
set.seed(1)
rows_a <- sample(1:nrow(survey), 15)   # pick 15 random row numbers (1 to 30 people)
sample_a <- survey[rows_a, ]           # keep those rows
mean(sample_a$self_efficacy)           # the mean of subsample A
#> [1] 33.93333
```

Now we take a **second** subsample, with a different seed:

```
set.seed(2)
rows_b <- sample(1:nrow(survey), 15)
sample_b <- survey[rows_b, ]
mean(sample_b$self_efficacy)           # the mean of subsample B
#> [1] 33
```

The two subsample means are **not the same**, and neither is exactly equal to the mean of the whole dataset:

```
mean(survey$self_efficacy)             # the full-data mean
#> [1] 33.23333
```

None of these numbers is "wrong". They simply show that a sample mean is an estimate that wobbles a bit. To assess that wobble, we first ask a bigger question: if we could take *many* samples, how would their means be spread out? That spread is characterized by the **sampling distribution**, and it is the key to everything that follows.

Your turn

Repeat the steps above for the `study_hours` variable. Draw two subsamples of 15 people (use `set.seed(1)` and `set.seed(2)`), and compare their two means with the full-data mean. Are the three numbers close together or far apart?

---

4 The sampling distribution

In the previous section, each subsample gave a slightly different mean. Now imagine repeating a whole study many times: each time you draw a fresh sample of the same size, assess it, and write down its mean. If you collected all those means together, they would form their own distribution. This is the **sampling distribution of the mean**: the distribution of the sample mean across many repeated samples.

With real data this is impossible: we collect our sample once, and that is it. The sampling distribution of a sample mean is a theoretical idea, but to make it concrete, we can **simulate** it. We assume an exemplary population with a mean and a standard deviation that we choose ourselves, draw many samples from it, and look at their means. This section uses code you can run and read; you do **not** need to write it yourself.

Step 1: one simulated sample. The function `rnorm(n, mean, sd)` draws `n` random values from a normal population with a given mean and standard deviation. We pretend the population has an average of 32 and a standard deviation of 6:

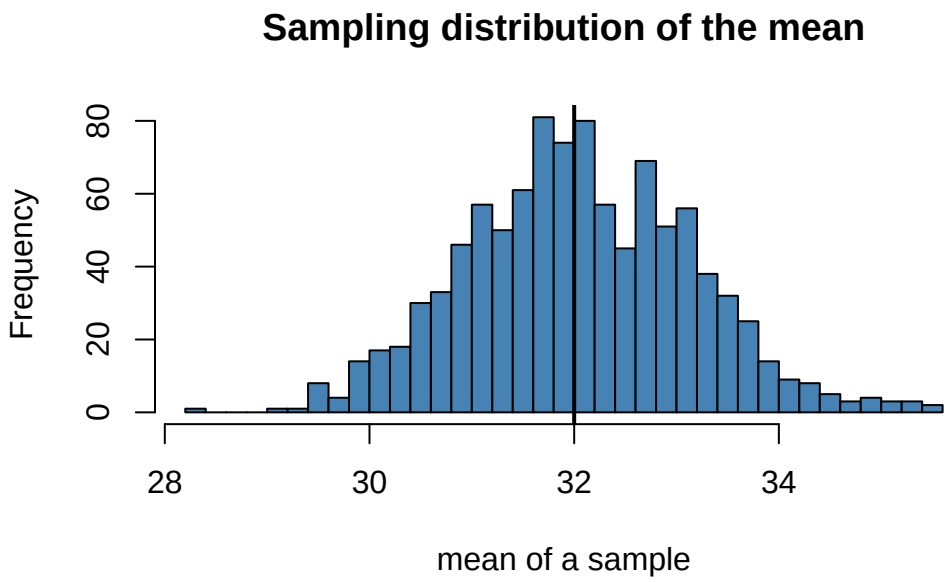
```
pop_mean <- 32   # the TRUE population mean (we decide it)
pop_sd   <- 6    # the TRUE population standard deviation
n        <- 30   # people per sample

set.seed(2026)
one_sample <- rnorm(n, mean = pop_mean, sd = pop_sd)
mean(one_sample)   # the mean of this one sample: close to 32, but not exactly
#> [1] 31.56543
```

Step 2: many sample means. Now we draw not one sample, but **1000** samples, and write down the **mean** of each. The function `replicate()` repeats an action many times. We then make a histogram of all those means:

```
set.seed(2026)
sample_means <- replicate(1000, mean(rnorm(n, mean = pop_mean, sd = pop_sd)))

hist(sample_means,
     breaks = 30,
     col = "steelblue",
     main = "Sampling distribution of the mean",
     xlab = "mean of a sample")
abline(v = pop_mean, col = "black", lwd = 2)   # the true population mean
```



This picture, the histogram of many sample means, **is** the sampling distribution. Three things are worth noticing:

1. It is **centered on the true population mean** (the black line at 32). Most samples give a mean near the truth.
2. It is **bell-shaped**: means near the center are common, means far away are rare.
3. Its **standard deviation** has a special name: the **standard error**. The standard error is simply how wide the sampling distribution is.

We can check point 3. The standard deviation of our 1000 means should be close to the population standard deviation divided by the square root of the sample size:

```
sd(sample_means)     # the standard deviation of the sampling distribution
#> [1] 1.106423
pop_sd / sqrt(n)     # the standard error, from the formula
#> [1] 1.095445
```

These two numbers are almost the same. So the abstract "standard error" is really something you can see: it is the **width of the sampling distribution**.

! Two different distributions – do not mix them up

It is easy to confuse two pictures:

- The **distribution of the data**: one histogram of all the scores in a single sample (for example, everyone's self-efficacy score). Each bar shows how many **people** had those scores.
- The **sampling distribution**: a histogram of the **means** of many different samples. Each bar shows how many **samples** had those mean values.

The first is about observed **people**; the second is about theoretical **sample means**. They are not the same.

Your turn

Change the sample size `n` from `30` to `100` and run Step 2 again. What happens to the **width** of the sampling distribution? Does `sd(sample_means)` get bigger or smaller? Write one sentence linking your answer to the idea of a "precise estimate".

Afterwards, set `n` back to `30` before you continue.

---

5 Confidence intervals by hand

A point estimate is a single number, but we just saw that it wobbles. So a single number alone does not tell the whole story. A **confidence interval** is a **range** around the point estimate.

The "95%" is a statement about our **method**, not about one single interval. Imagine taking many samples from the population and building a **95% confidence interval** from each one. In the long run, about **95%** of those intervals would contain the true population value.

i This idea is hard, and that is normal

The confidence interval is one of the **most difficult ideas** in this whole course. **Fully understanding it takes time**, so do not worry if it feels slippery right now. We will come back to this idea again and again throughout the course, and each time a bit more will fall into place. Later in this tutorial, when we simulate many confidence intervals, you will be able to see this happen with your own eyes.

5.1 Visualisation

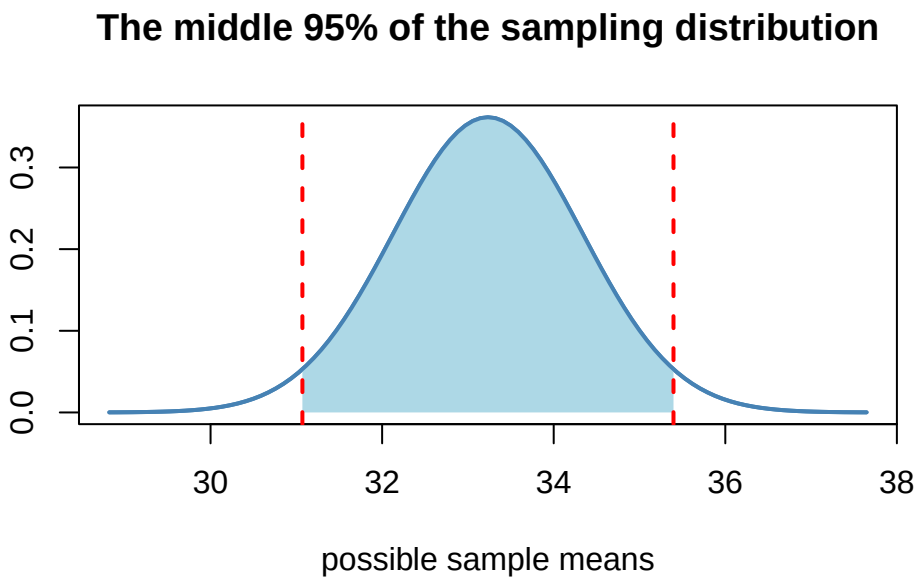
We build the interval from the sampling distribution you just saw. Remember: the sampling distribution is bell-shaped, and **95%** of all sample means land close to the center. "Close" turns out to mean **within 1.96 standard errors** of the center. The picture below shows this middle 95% shaded:

```
m  <- mean(survey$self_efficacy)                      # our point estimate
se <- sd(survey$self_efficacy) / sqrt(nrow(survey))  # our standard error

# draw the sampling distribution as a smooth bell curve
curve(dnorm(x, mean = m, sd = se),
      from = m - 4 * se, to = m + 4 * se,
      lwd = 2, col = "steelblue",
      xlab = "possible sample means", ylab = "",
      main = "The middle 95% of the sampling distribution")

# shade the middle 95%: within 1.96 standard errors of the center
xs <- seq(m - 1.96 * se, m + 1.96 * se, length.out = 200)
polygon(c(xs, rev(xs)),
        c(dnorm(xs, m, se), rep(0, length(xs))),
        col = "lightblue", border = NA)

curve(dnorm(x, mean = m, sd = se), add = TRUE, lwd = 2, col = "steelblue")
abline(v = c(m - 1.96 * se, m + 1.96 * se), col = "red", lwd = 2, lty = 2)  # the 95% edges
```



The two red dashed lines sit exactly **1.96 standard errors** from the center. Everything between them is the middle **95%**. This is where the number 1.96 comes from: it marks the edges of the middle 95% of a bell-shaped distribution.

Now the clever step. Because a sample mean lands within 1.96 standard errors of the truth **95% of the time**, if we reach out 1.96 standard errors on each side of *our* sample mean, our range will reach back to the true value 95% of the time. That range is the confidence interval.

5.2 Building the interval step by step

We build the interval using R as a calculator. We need four ingredients.

1. The point estimate (the sample mean):

```
m <- mean(survey$self_efficacy)
m
#> [1] 33.23333
```

2. The standard error. This is the width of the sampling distribution from the previous section. We estimate it from our single sample as the standard deviation divided by the square root of the number of people. This uses the function `sqrt()` (square root):

```r
s  <- sd(survey$self_efficacy)   # standard deviation
n  <- nrow(survey)               # number of people
se <- s / sqrt(n)                # standard error
se
#> [1] 1.103478
```

> **i** A smaller standard error means a more precise estimate
>
> Look at the formula `s / sqrt(n)`: a **bigger sample** (larger `n`) makes the standard error **smaller**. More people means a narrower sampling distribution, less wobble, and a more precise estimate. This is one reason why larger samples are better, and it is exactly what you saw in the Your turn of the previous section.

3. The z-value. For a 95% confidence interval, we multiply the standard error by the fixed number we just met in the picture: **1.96**. This number comes from the tail areas under the standard normal distribution, assuming we know the population's standard deviation (see the lecture for more details). Other confidence levels, such as 80% or 99%, need a different z-value; we come back to this in the section *Other confidence levels* below. We set our z-value:

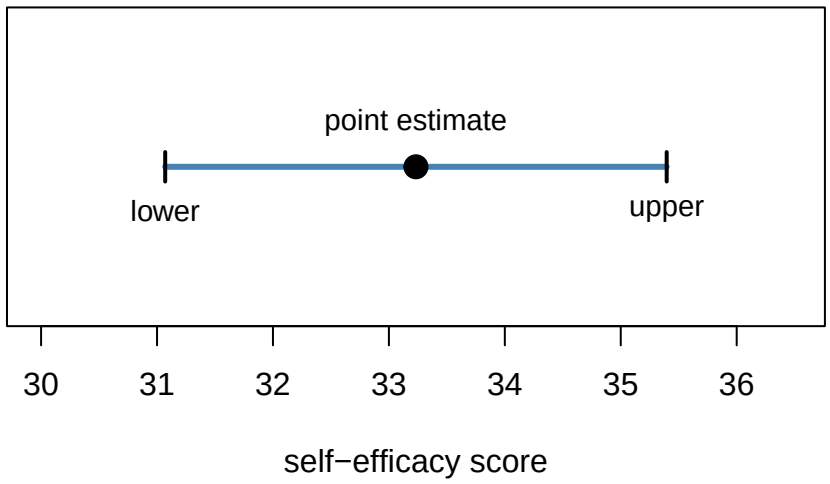
```r
z <- 1.96
```

4. The interval. The lower and upper edges are the point estimate, minus and plus `z * se`:

```r
lower <- m - z * se
upper <- m + z * se
c(lower, upper)                  # the 95% confidence interval
#> [1] 31.07052 35.39615
```

We can draw this interval as a range around the point estimate:

```r
plot(NULL,
     xlim = c(lower - se, upper + se), ylim = c(0.5, 1.5),
     yaxt = "n", ylab = "",
     xlab = "self-efficacy score",
     main = "The confidence interval as a range around the estimate")
segments(lower, 1, upper, 1, lwd = 3, col = "steelblue")            # the interval
segments(c(lower, upper), 0.95, c(lower, upper), 1.05, lwd = 2)     # end ticks
points(m, 1, pch = 19, cex = 1.6)                                   # the point estimate
text(m, 1.15, "point estimate", cex = 0.9)
text(lower, 0.85, "lower", cex = 0.9)
text(upper, 0.85, "upper", cex = 0.9)
```



The dot in the middle is our best guess (the point estimate). The blue line is the plausible range for the population average. Remember what the 95% means: it is not that this one interval has a 95% probability of being right, but that intervals built this way catch the true value about 95% of the time.

> ⚠ Watch out: two common misreadings
>
> Two things a 95% confidence interval does **not** mean:
>
> - It does **not** mean "95% of people fall in this range." The interval is about the **average**, not about individual people.
> - It does **not** mean "there is a 95% chance the population average is inside *this* interval." The 95% is about the **method**, not about this one interval: intervals built this way contain the population average about 95% of the time.

Your turn

Compute a 95% confidence interval **by hand** for the `study_hours` variable, following the same four steps: the mean, the standard error (`sd / sqrt(n)`), the z-value `1.96`, and the lower and upper bounds Write down the interval.

5.3 Other confidence levels

So far we always used **95%**. But 95% is a choice, not a rule. The **confidence level** (in the lecture: **C**) can also be, for example, 80%, 90%, or 99%. The level decides which **z-value** we use.

For a C% interval we want the **middle C%** of the sampling distribution. What is left over is split equally over the two tails. For 95%, that is 2.5% in each tail; for 80%, it is 10% in each tail.

Confidence level (C)	Left over in each tail	z-value
80%	10%	1.28
90%	5%	1.645
95%	2.5%	1.96
99%	0.5%	2.576

On paper, you look up the z-value in a table (see the lecture). In R, the function `qnorm()` finds it for you. `qnorm()` gives the z-value that has a given area to its **left**. For a 95% interval, 2.5% is left over in the upper tail, so 97.5% of the area lies to the left of z:

```
qnorm(0.975)   # 95%: 2.5% in the upper tail
#> [1] 1.959964
qnorm(0.90)    # 80%: 10% in the upper tail
#> [1] 1.281552
qnorm(0.995)   # 99%: 0.5% in the upper tail
#> [1] 2.575829
```

Now we build an 80%, a 95%, and a 99% interval for self-efficacy. Only the z-value changes; the point estimate and the standard error stay the same:

```
m  <- mean(survey$self_efficacy)
se <- sd(survey$self_efficacy) / sqrt(nrow(survey))

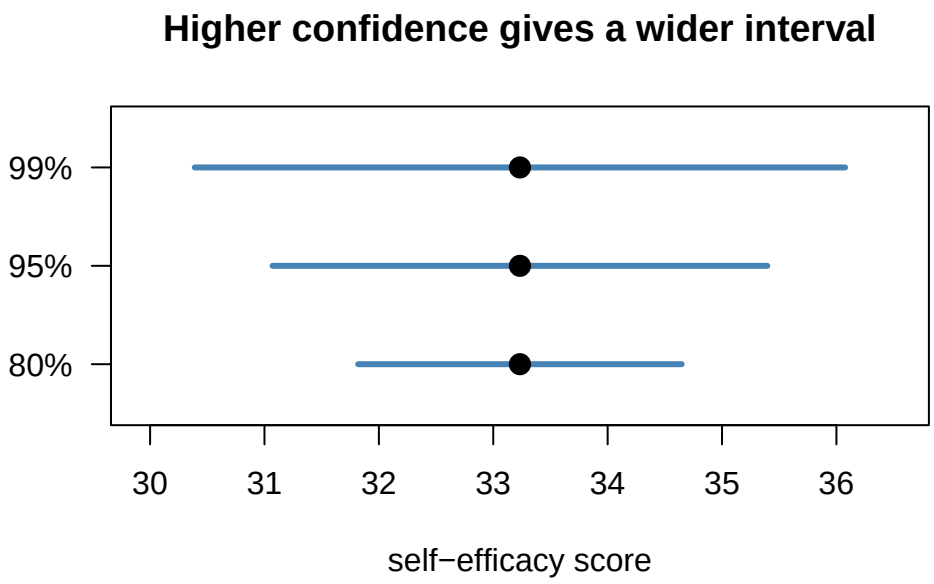
z80 <- qnorm(0.90)
z95 <- qnorm(0.975)
z99 <- qnorm(0.995)

c(m - z80 * se, m + z80 * se)   # 80% confidence interval
#> [1] 31.81917 34.64750
c(m - z95 * se, m + z95 * se)   # 95% confidence interval
#> [1] 31.07056 35.39611
```

```r
c(m - z99 * se, m + z99 * se)   # 99% confidence interval
#> [1] 30.39096 36.07570
```

We draw the three intervals above each other:

```r
zs <- c(z80, z95, z99)
plot(NULL,
     xlim = c(m - 3 * se, m + 3 * se), ylim = c(0.5, 3.5),
     yaxt = "n", ylab = "",
     xlab = "self-efficacy score",
     main = "Higher confidence gives a wider interval")
segments(m - zs * se, 1:3, m + zs * se, 1:3, lwd = 3, col = "steelblue")
points(rep(m, 3), 1:3, pch = 19, cex = 1.4)
axis(2, at = 1:3, labels = c("80%", "95%", "99%"), las = 1)
```



The higher the confidence level, the **wider** the interval. This is a trade-off: if you want to be more confident that your method catches the population average, you have to accept a wider, less precise range. The meaning stays the same as before: an 80% interval is built in a way that catches the population average in about **80%** of the samples.

Your turn

Compute a **90%** and a **99%** confidence interval by hand for the `study_hours` variable. Use `qnorm()` to find the two z-values. Compare them with your 95% interval from before: which interval is the widest, and why?

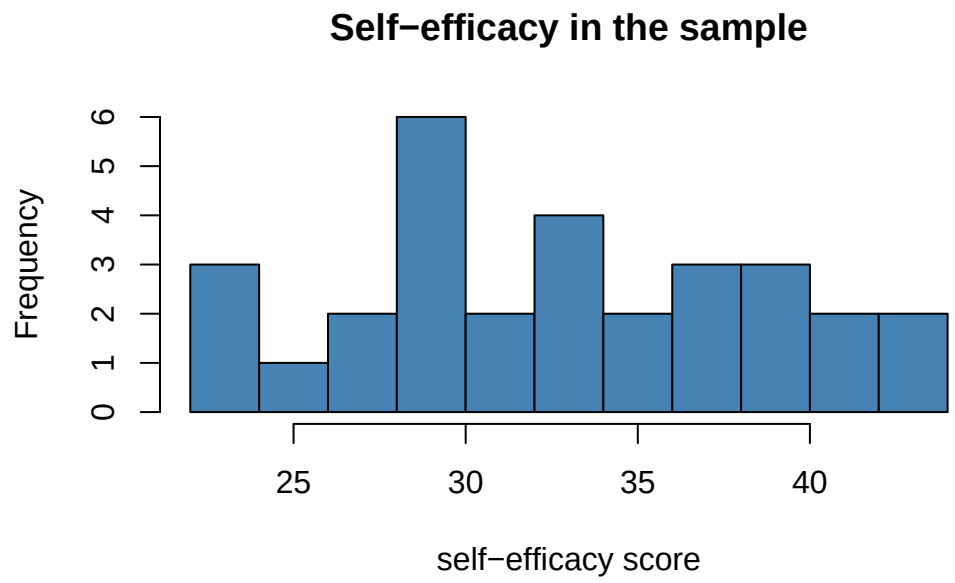
---

6 A confidence interval for your own variable

From now on you work with the variable(s) you chose for your research question. This is the variable whose average you want to estimate: your **dependent variable**. We do three things: look at it, estimate it, and interpret it.

Step 1: visualise the distribution. Always look at the variable first, with a histogram (from Week 2):

```
hist(survey$self_efficacy,
     breaks = 10,
     col = "steelblue",
     main = "Self-efficacy in the sample",
     xlab = "self-efficacy score")
```



Which distribution is this?

This histogram is the **distribution of the data**: each bar shows how many **people** had those self-efficacy scores. It is **not** the sampling distribution from the section above, which was about the means of many samples. Keep the two apart.

Step 2: the point estimate and the confidence interval. These are the same four steps as the section above, gathered in one place:

```
m  <- mean(survey$self_efficacy)
s  <- sd(survey$self_efficacy)
n  <- nrow(survey)
se <- s / sqrt(n)

point_estimate <- m
ci_lower <- m - 1.96 * se
ci_upper <- m + 1.96 * se

point_estimate
#> [1] 33.23333
c(ci_lower, ci_upper)
#> [1] 31.07052 35.39615
```

Step 3: interpret. Two questions matter here.

- *What does the point estimate say?* It describes your group. For example, a mean self-efficacy of 33.2 on this scale tells you how self-efficacious the students in the sample are, on average (high, medium, or low, depending on the scale).
- *What does the confidence interval add?* It tells you how **precise** that estimate is. A **narrow** interval means a precise estimate; a **wide** interval means there is more uncertainty about the population average.

Going further: the t-distribution and t.test()

The value **1.96** comes from the **normal distribution** (also called the **z-distribution**). It assumes we know the standard deviation in the population very well. In real research we only *estimate* the standard deviation from our sample, so for **small samples** the correct multiplier is a little **larger** than 1.96. That larger multiplier comes from the **t-distribution**.
You learned how to compute it by hand already. In R there is the function, `t.test()`, that builds the interval for you using the t-distribution:

```r
t.test(survey$self_efficacy)$conf.int
#> [1] 30.97647 35.49020
#> attr(,"conf.level")
#> [1] 0.95
```

Compare this with your by-hand interval: it is very **similar**, but usually a little **wider**, because the t-distribution is more cautious with small samples. As the sample gets bigger, the t-value gets closer and closer to 1.96, and the two intervals become essentially identical when the sample size gets very large.
For another confidence level, add the argument `conf.level`, for example an 80% interval:

```r
t.test(survey$self_efficacy, conf.level = 0.80)$conf.int
#> [1] 31.78620 34.68047
#> attr(,"conf.level")
#> [1] 0.8
```

Your turn

For your **own** dependent variable:

1. Make a histogram with a clear title and x-axis label.
2. Compute the point estimate and the 95% confidence interval.
3. Write two sentences: one about what the point estimate says about your group, and one about what the confidence interval adds.

---

7 Does the interval cover the true mean?

This is a good moment to test the idea. Let us pretend, for a second, that the mean of the **whole dataset** is the "true" population value. Then we take a **subsample**, build its confidence interval (assuming that we know the population standard deviation), and check: does the full-data mean fall **inside** that interval?

```r
true_mean <- mean(survey$self_efficacy)   # we treat this as the "truth"
pop_sd   <- 6 # the populaiton standard deviation

set.seed(1)
rows <- sample(1:nrow(survey), 15)
```

```r
sub  <- survey[rows, ]

m  <- mean(sub$self_efficacy)
n  <- nrow(sub)
se <- pop_sd / sqrt(n)

low  <- m - 1.96 * se
high <- m + 1.96 * se
c(low, high)
#> [1] 30.89691 36.96975

# is the true mean inside this interval? (TRUE or FALSE)
low <= true_mean & true_mean <= high
#> [1] TRUE
```

The last line asks: "is `true_mean` greater than or equal to the lower edge **and** smaller than or equal to the upper edge?" It gives back `TRUE` if the interval **catches** the true mean, and `FALSE` if it misses.

If you change the seed and run it again, you will sometimes get `TRUE` and sometimes `FALSE`. A confidence interval does not catch the truth every single time; a 95% interval is built to catch it about 95% of the time in the long run.

Your turn

Run the check above three times, each time with a different seed (for example `set.seed(3)`, `set.seed(4)`, `set.seed(5)`). For each run, note whether the interval caught the full-data mean (`TRUE` or `FALSE`). How many of your three intervals caught it?

---

8 Simulating many intervals

In the earlier section we simulated many sample **means** and saw the sampling distribution. Now we go one step further: for many samples we build a whole **confidence interval** each time, and count how often the interval catches the true population mean. Because *we* chose the true mean, we can check this exactly. As before, this is code to run and read.

We use the same pretend population as before (mean 32, standard deviation 6, samples of 30):

```r
pop_mean <- 32
pop_sd   <- 6
n        <- 30
n_sim    <- 100

set.seed(2026)
# For each sample, compute its lower and upper CI edge.
sims <- replicate(n_sim, {
  x  <- rnorm(n, mean = pop_mean, sd = pop_sd)
  m  <- mean(x)
  se <- pop_sd / sqrt(n)
  c(lower = m - 1.96 * se, upper = m + 1.96 * se)
})

lowers <- sims["lower", ]
uppers <- sims["upper", ]

# Does each interval contain the true mean?
contains <- lowers <= pop_mean & pop_mean <= uppers
mean(contains)   # the share of intervals that caught the truth
#> [1] 0.92
```

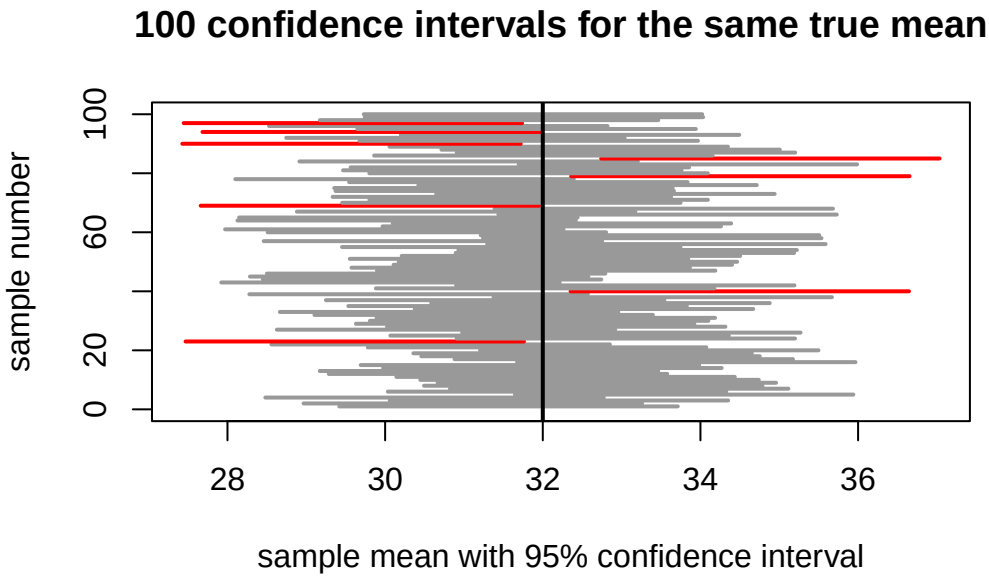
That last number is the **coverage**: the share of intervals that caught the true mean. It should be close to **0.95** (95%). This is what "95% confidence" really means.

Finally, we draw each interval as a horizontal line, stacked on top of each other. The vertical black line is the true population mean. Intervals that **miss** the true mean are drawn in **red**:

```r
plot(NULL,
     xlim = range(c(lowers, uppers)),
     ylim = c(0, n_sim),
     xlab = "sample mean with 95% confidence interval",
     ylab = "sample number",
     main = "100 confidence intervals for the same true mean")

# one horizontal line per interval; red if it misses the true mean
segments(x0 = lowers, y0 = 1:n_sim,
         x1 = uppers, y1 = 1:n_sim,
         col = ifelse(contains, "grey60", "red"),
         lwd = 2)

abline(v = pop_mean, col = "black", lwd = 2)   # the true population mean
```



This picture is the definition of a 95% confidence interval, made visible: build many intervals, and about 95% of them cross the true value. **Most** intervals cross the black line (they caught the truth), and a **few red ones** miss it. About 5 out of 100 are expected to miss. That is the price of 95% confidence: it is right most of the time, but not always.

Your turn

Change the sample size `n` near the top from `30` to `100`, then run the code again.

1. What happens to the **width** of the intervals in the plot?
2. Does the **coverage** (the share close to 0.95) change much?

Write one sentence explaining what a larger sample does to your estimate.

Then set `n` back to `30`, and replace both `1.96` in the code by `qnorm(0.90)` (an **80%** interval). Run the code again. What happens to the width of the intervals, to the number of red intervals, and to the coverage?

---

9 Recap: what you can now do

Last week you described a single sample. This week you learned to reason from that sample to the population. You can now:

- explain the difference between a **sample** and a **population**;
- describe a sample mean as a **point estimate** of the population mean;
- explain **sampling fluctuation** by drawing subsamples with `set.seed()` and `sample()`;
- describe the **sampling distribution** of the mean, and know that its standard deviation is the **standard error**;
- compute a 95% **confidence interval by hand** using the mean, the standard error (`sd / sqrt(n)`), and the z-value `1.96`, and explain where the `1.96` comes from;
- compute confidence intervals for **other confidence levels** (such as 80% and 99%) with `qnorm()`, and explain why a higher level gives a wider interval;
- visualise, estimate, and interpret a confidence interval for your own variable;
- check whether an interval catches a known mean with a logical test;
- simulate many samples with `rnorm()` and `replicate()` and see that about 95% of the intervals catch the true mean.

A small cheat sheet to keep:

You want…	You write…
Point estimate (sample mean)	`mean(df$x)`
Standard deviation	`sd(df$x)`
Number of people	`nrow(df)`
Make the random choice repeatable	`set.seed(1)`
Pick random row numbers	`sample(1:nrow(df), 15)`
Standard error	`sd(df$x) / sqrt(nrow(df))`
Lower edge of 95% CI	`m - 1.96 * se`
Upper edge of 95% CI	`m + 1.96 * se`
z-value for another level (here 80%)	`qnorm(0.90)`
CI with the t-distribution	`t.test(df$x)$conf.int`
CI with the t-distribution, other level	`t.test(df$x, conf.level = 0.80)$conf.int`
Is a value inside an interval?	`low <= value & value <= high`
One random sample	`rnorm(n, mean = , sd = )`
Many sample means	`replicate(1000, mean(rnorm(n, mean = , sd = )))`
Share that is TRUE	`mean(contains)`

Well done. Keep your materials in the `research_methods` folder, we build directly on them next week.